

회의록을 정리하는

AI 서기

회의록을 정리하는 AI 서기

1. 음성을 텍스트로 변환하기
2. 로컬에서 음성을 텍스트로 변환하기
3. 문장과 화자 구분하기
4. 회의록을 정리하는 AI 서기 완성하기

1. 음성을 텍스트로 변환하기

- GPT는 텍스트를 이해하고 처리하는 언어 모델이므로 녹음된 음성 파일을 텍스트로 변환해서 제공해야만 처리할 수 있음
- STT(Speech-To-Text)
 - 음성을 텍스트로 변환하는 기술
 - 오픈AI는 위스퍼(whisper) API를 통해 이 기능을 제공

1. 음성을 텍스트로 변환하기

- (실습) 위스퍼 API 활용하기

- 가상 환경 설정

```
# 가상 환경 생성
$ python -m venv .venv

# 윈도우 가상 환경 활성화
$ source .venv/Scripts/activate # 맥 환경에서는 Scripts를 bin으로 변경
(.venv)
$ python -m pip install --upgrade pip
```

- 배포된 requirements.txt 파일을 대상으로 다음 명령 실행하여 패키지 설치

```
# chap04에 설치했던 패키지 설치
(.venv)
$ pip install -r requirements.txt
$ pip list          # 설치된 패키지들의 버전 확인 해 볼 것
```

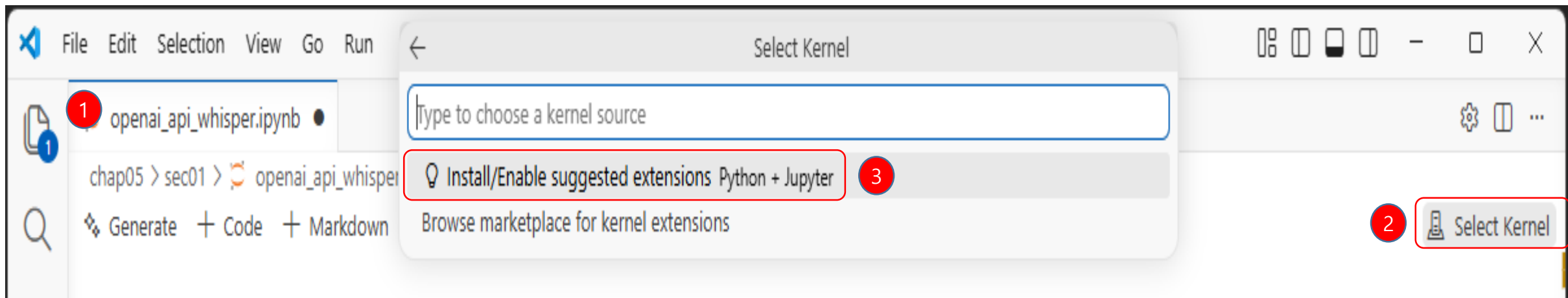
- 이번 chapter는 패키지별 상호 호환성이 중요
- 최신 버전의 패키지들로 설치 시 error 발생

1. 음성을 텍스트로 변환하기

- (실습) 위스퍼 API 활용하기

- 주피터 노트북 사용하기 - 단계별 기능을 익히기 위해

1. chap05/sector01/sec01/ 폴더를 만들고 그 안에 openai_api_whisper.ipynb 파일 만들기
2. ipynb 파일을 처음 사용한다면 Select Kernel 버튼을 클릭
3. 표시되는 화면에서 파이썬과 주피터를 사용할 수 있는 extensions을 설치

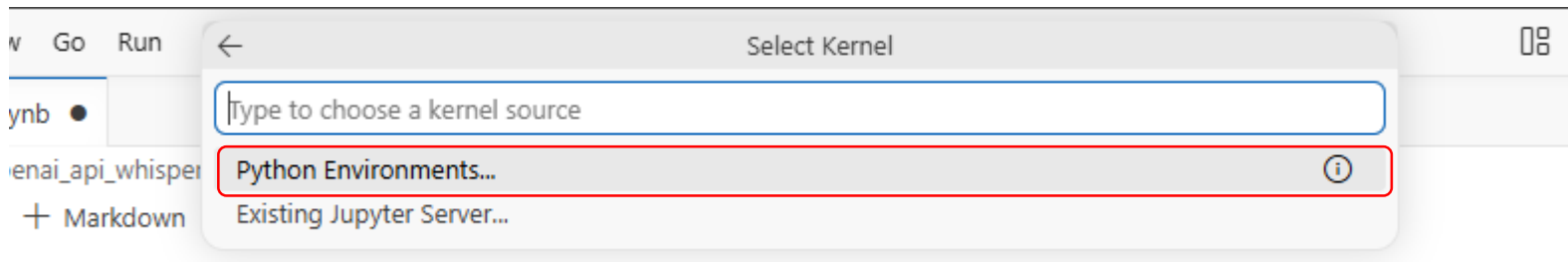


1. 음성을 텍스트로 변환하기

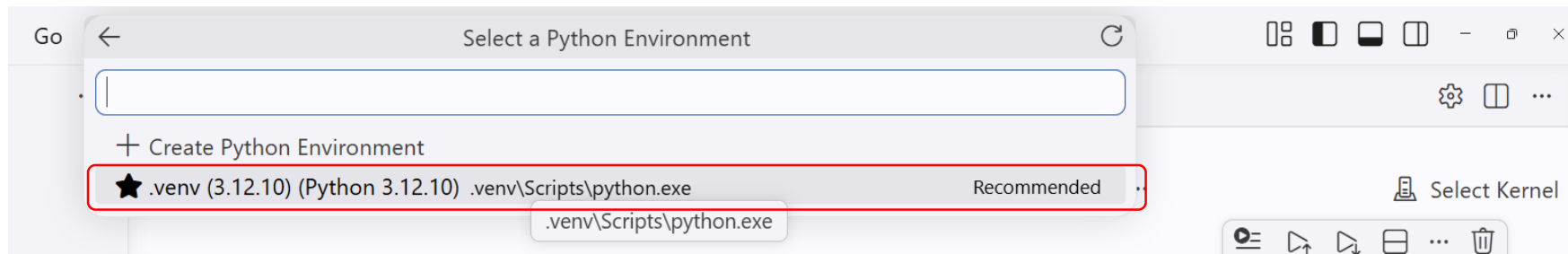
- (실습) 위스퍼 API 활용하기

- 주피터 노트북 사용하기 - 단계별 기능을 익히기 위해

4. extensions 을 설치하면 파이썬 환경을 선택하는 메뉴가 나옴. 여기서 Python Environments... 를 클릭



5. 현재 프로젝트 폴더의 가상 환경인 .venv를 선택

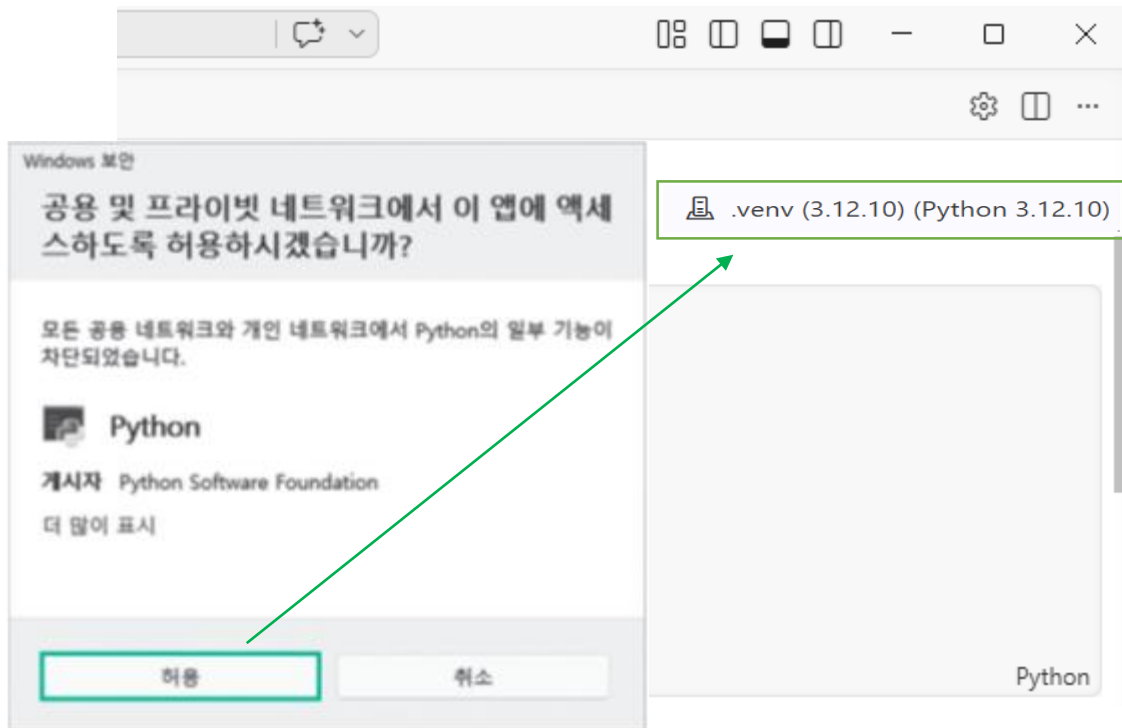


1. 음성을 텍스트로 변환하기

- (실습) 위스퍼 API 활용하기

- 주피터 노트북 사용하기 - 단계별 기능을 익히기 위해

5-1. 주피터 노트북을 위한 커널을 처음 선택한다면 보안 경고가 나올 수 있고, [허용]을 클릭하면 주피터 노트북에서 사용할 가상 환경 `.venv` 가 자동으로 선택

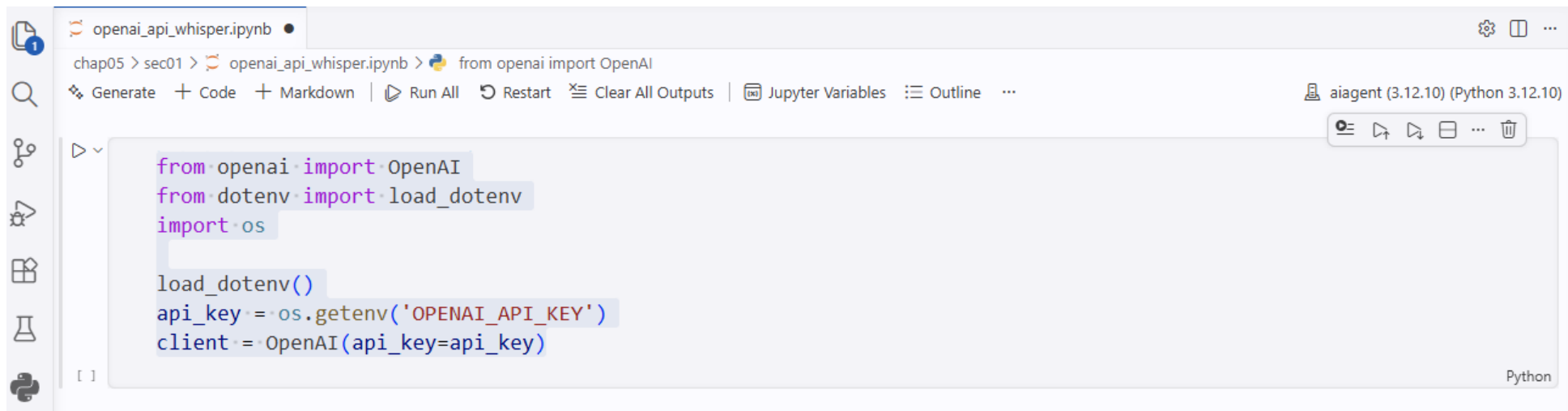


1. 음성을 텍스트로 변환하기

- (실습) 위스퍼 API 활용하기

- 주피터 노트북 사용하기 - 단계별 기능을 익히기 위해

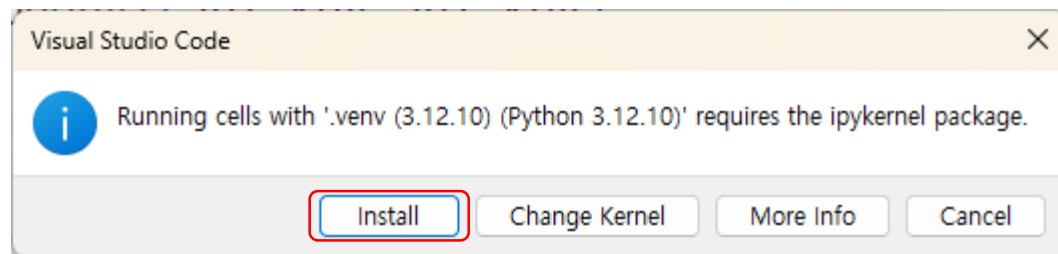
6. [+Code] 버튼을 클릭하면 나오는 셀에 테스트를 위한 파이썬 코드 입력 후  실행



```
from openai import OpenAI
from dotenv import load_dotenv
import os

load_dotenv()
api_key = os.getenv('OPENAI_API_KEY')
client = OpenAI(api_key=api_key)
```

ps) 실행 시 아래 팝업창이 뜰 경우, install 버튼 클릭해 설치



1. 음성을 텍스트로 변환하기

- (실습) 위스퍼 API 활용하기

- 주피터 노트북 사용하기 - 단계별 기능을 익히기 위해

7. 셀의 왼쪽 아래에 넘버링이 표시되고, 체크 표시가 생기면 셀 실행이 성공했다는 의미



The image shows a Jupyter Notebook cell with a light gray background. On the left side of the cell, there is a vertical blue bar. To the right of the bar, the Python code is displayed in a monospaced font with syntax highlighting. The code imports OpenAI and dotenv, loads environment variables, and creates an OpenAI client. At the bottom left of the cell, there is a status bar with a red border. It contains the text '[1]' in a gray box, a green checkmark, and '0.7s'. On the far right of the cell, the word 'Python' is written in a small font.

```
from openai import OpenAI
from dotenv import load_dotenv
import os

load_dotenv()
api_key = os.getenv('OPENAI_API_KEY')
client = OpenAI(api_key=api_key)
```

[1] ✓ 0.7s Python

1. 음성을 텍스트로 변환하기

- (실습) 위스퍼 API 활용하기

- MP3 파일을 텍스트로 변환하기

1. 오픈AI API를 사용하기 위해 client를 선언

```
from openai import OpenAI
from dotenv import load_dotenv
import os

load_dotenv()
api_key = os.getenv('OPENAI_API_KEY')
client = OpenAI(api_key=api_key)
```

1. 음성을 텍스트로 변환하기

- (실습) 위스퍼 API 활용하기

- MP3 파일을 텍스트로 변환하기

2. 오픈AI의 STT 관련 문서를 참고하여 다음처럼 코드 작성

- 오픈AI의 STT 관련 문서: <https://platform.openai.com/docs/guides/speech-to-text>

```
audio_file_path = '../audio/lsy_audio_2023_58s.mp3' # MP3 파일 경로 입력

with open(audio_file_path, 'rb') as audio_file:
    transcription = client.audio.transcriptions.create(
        model="whisper-1",
        file=audio_file
    )

transcription
```

1. 음성을 텍스트로 변환하기

- (실습) 위스퍼 API 활용하기

- MP3 파일을 텍스트로 변환하기

- 2. 오픈AI의 STT 관련 문서를 참고하여 다음처럼 코드 작성

- 셀 실행 후 음성 파일 텍스트 출력

```
Transcription(text='안녕하세요. 이 강의는 GPT API로 챗봇 만들기라는 내용을 다루는 강의입니다. GPT API에 대해서 생소하신 분들도 있을 텐데 우리가 잘 알고 있는 채GPT, 채GPT 기능을 이용해서 우리가 원하는 프로그램을 어떻게 만드는지에 대해서 이야기할 거예요. 그래서 뭐 이런 강의들이 사실 많이 있습니다. 그래서 여러 가지들이 있는데 좀 이 강의의 특징이라고 한다면 GPT로 명확한 미션을 달성하는 챗봇 프로그램을 만드는 게 사실 쉽지는 않은데 이걸 어떻게 해서 구현을 하는지 그리고 그게 왜 필요한지에 대해서 좀 이야기를 할 거고요. 그 예제로 예제는 여러 가지가 될 수 있는데 여기서 예제로 하는 것은 음악 플레이리스트 동영상상을 자동으로 대화를 통해서 생성하는 프로그램 만드는 것을 다루려고 합니다. 그래서 프로그램이 실행되는 모습을 한번 보여드릴게요. 우리가 만들 프로그램은 이런 식으로 이제 나타나게 되고', logprobs=None, usage=UsageDuration(seconds=58.0, type='duration'))
```

1. 음성을 텍스트로 변환하기

- (실습) 위스퍼 API 활용하기

- MP3 파일을 텍스트로 변환하기

3. 위스퍼 API는 번역 기능도 제공

- 한국어 음성 파일을 영어로 바로 번역하기

```
audio_file_path = '../audio/lsy_audio_2023_58s.mp3' # MP3 파일 경로 입력

with open(audio_file_path, 'rb') as audio_file:
    transcription = client.audio.translations.create(
        model="whisper-1",
        file=audio_file,
    )

transcription
```

1. 음성을 텍스트로 변환하기

- (실습) 위스퍼 API 활용하기

- MP3 파일을 텍스트로 변환하기

3. 위스퍼 API는 번역 기능도 제공

- 셀 실행 후 음성 파일 텍스트 출력

Translation(text="Hello, this is a lecture on how to make a chatbot with GPT API. Some of you may be unfamiliar with GPT API. We're going to talk about how to make the program we want using the chat GPT function that we know well. So there are a lot of lectures like this. There are a lot of things, but if I were to say the characteristics of this lecture, it's not easy to make a chatbot program that achieves a clear mission with GPT. I'm going to talk about how to implement this and why it's necessary. As an example, there can be many examples. The example here is to create a program that automatically creates a music playlist video through conversation. I'll show you how the program runs. This is how the program we're going to make appears.")

2. 로컬에서 음성을 텍스트로 변환하기

- 위스퍼 API 사용시

- 장점

- 음성 파일의 내용이 오픈AI 서버로 전송되어 처리된 후 반환되므로 컴퓨터의 성능에 관계없이 간단한 코드 몇 줄로 기능 구현이 가능

- 단점

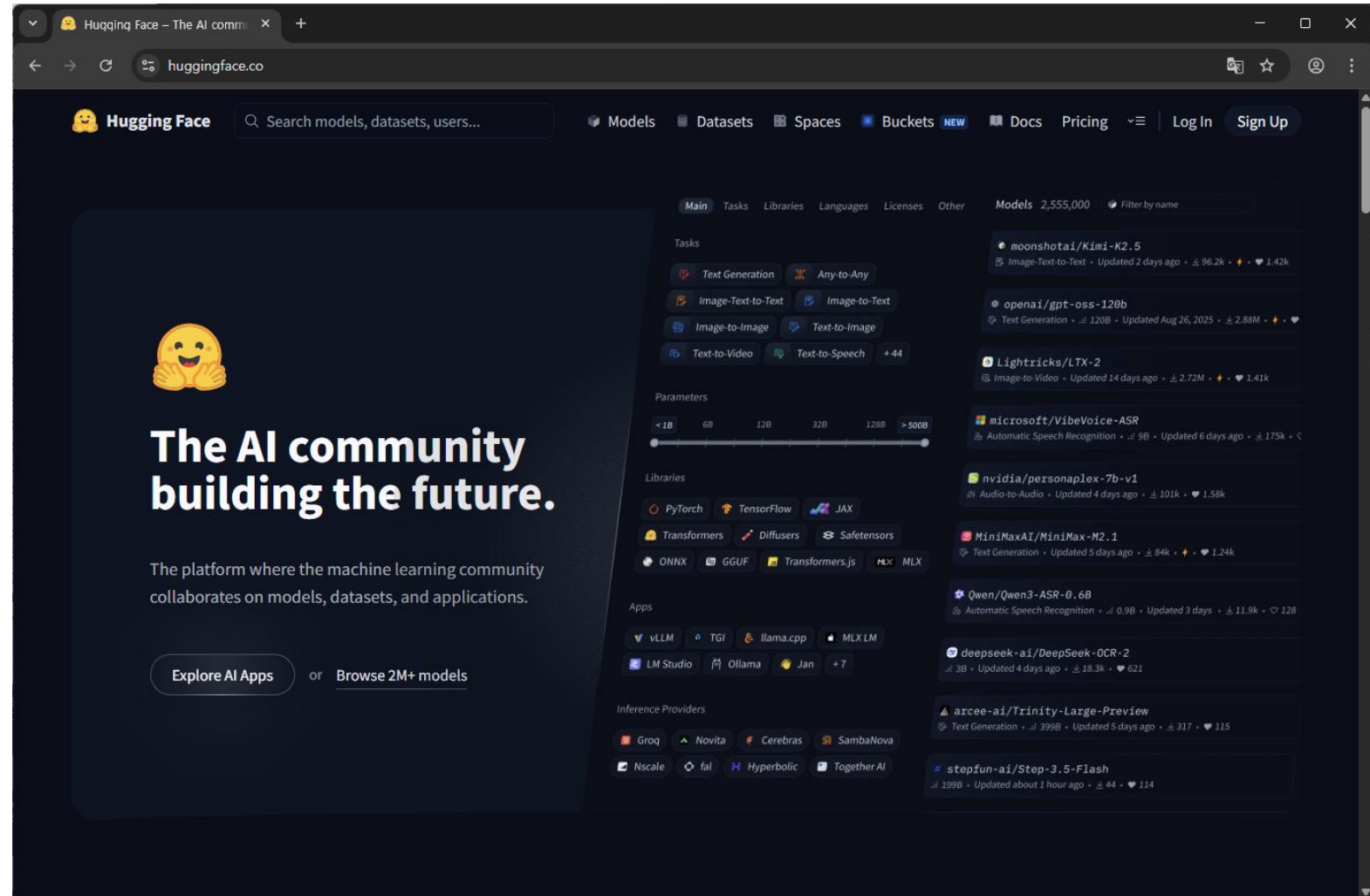
- 회의 내용이 회사 밖으로 유출되면 안되거나, 음성 파일의 길이가 길면 처리 비용이 많이 발생할 수 있음

- 위스퍼 API 사용시 단점을 극복하기 위해 로컬 환경에서 언어 모델을 내려 받아 처리할 수 있음

2. 로컬에서 음성을 텍스트로 변환하기

- 허깅페이스(HuggingFace)

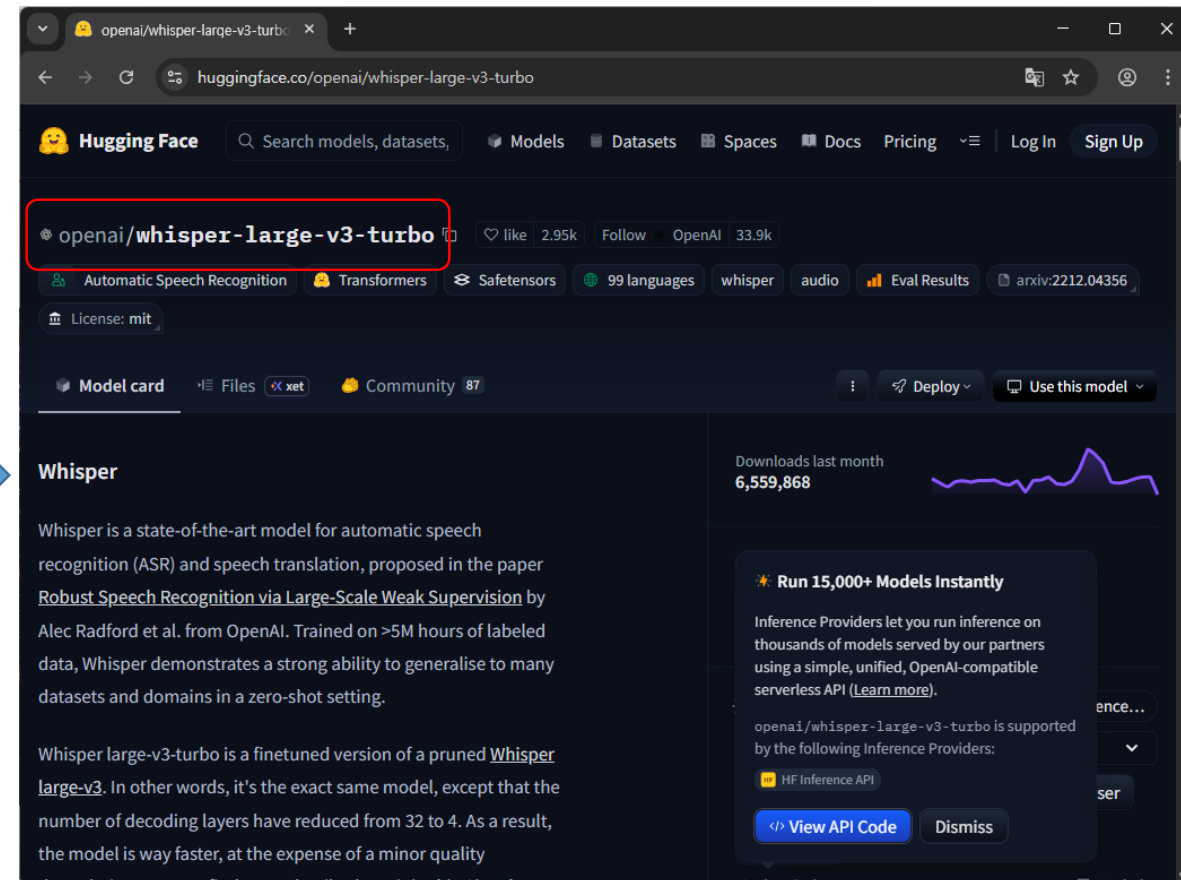
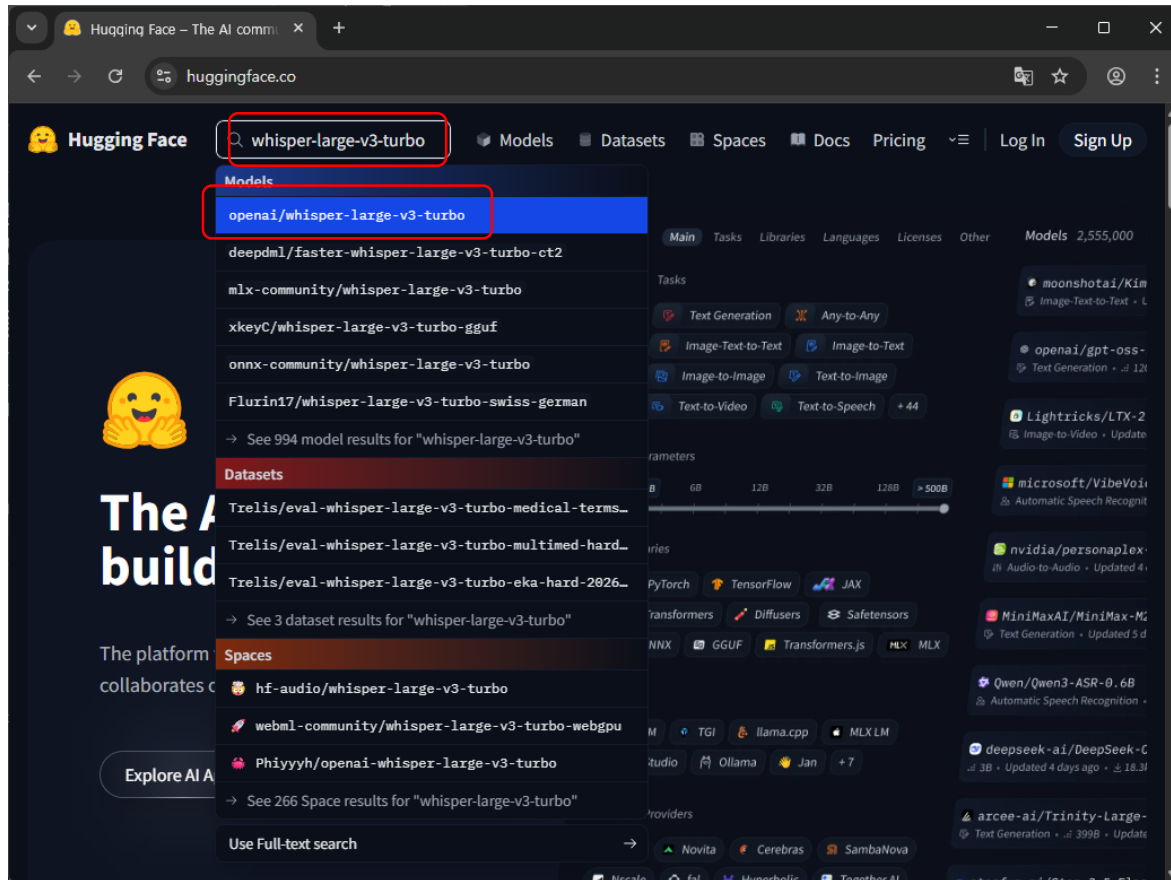
- 인공지능 모델을 개발하는 회사로,
허깅페이스 플랫폼 서비스로 더욱 잘 알려져
있음
- 전 세계의 수많은 개발자와 연구자가 만든
인공지능 모델, 데이터셋, 문서를 서비스
- <https://huggingface.co>



2. 로컬에서 음성을 텍스트로 변환하기

● 허깅페이스

- 앞선 실습에서 활용한 오픈AI에서 API로 제공하는 위스퍼 모델 역시 허깅페이스에 공개되어 있음
- 왼쪽 위 검색 창에서 'whisper'를 검색하면, whisper-large-v3-turbo가 보임



2. 로컬에서 음성을 텍스트로 변환하기

- (실습) 위스퍼 모델을 내려 받아 로컬에서 사용하기
 - 허깅페이스의 문서를 참조해 필요한 패키지 설치
 - 주의) 앞서 1절에서 모든 패키지 설치하는 이미 완료하였고, 아래 내용은 설치 버전 확인 용도로 살펴만 볼 것
 - 절대 다시 설치 하지 말 것

```
(.venv)
```

```
# $ pip install torch torchvision torchaudio
```

```
# $ pip install transformers accelerate
```

2. 로컬에서 음성을 텍스트로 변환하기

- (실습) 위스퍼 모델을 내려 받아 로컬에서 사용하기
 - 파일명 huggingface_whisper. ipynb 파일 만들고, 코드 작성

```
import torch
import librosa
from transformers import AutoModelForSpeechSeq2Seq, AutoProcessor, pipeline

device = "cuda:0" if torch.cuda.is_available() else "cpu"
torch_dtype = torch.float16 if torch.cuda.is_available() else torch.float32

model_id = "openai/whisper-small"

model = AutoModelForSpeechSeq2Seq.from_pretrained(
    model_id,
    torch_dtype=torch_dtype,
    low_cpu_mem_usage=True,
    use_safetensors=True
)

processor = AutoProcessor.from_pretrained(model_id)

... ( 생략 ) ...
```

2. 로컬에서 음성을 텍스트로 변환하기

- (실습) 위스퍼 모델을 내려 받아 로컬에서 사용하기

- 코드 설명

- `AutoModelForSpeechSeq2Seq` : 음성을 텍스트로 바꾸는 AI(예: whisper)
- `.from_pretrained()` : 이미 학습된 모델/도구를 다운로드해서 가져오는 함수
- 파라미터
 - `model_id = "openai/whisper-large-v3-turbo"` : 어떤 모델을 가져올지 이름 지정
 - `torch_dtype` : 숫자를 어떤 방식으로 저장할지 (속도 vs 정확도 조절)
 - `low_cpu_mem_usage=True` : RAM 적게 쓰면서 불러오기
 - `use_safetensors=True` : 안전한 형식으로 모델을 불러오도록 강제
- `AutoProcessor` : 데이터를 모델이 이해할 수 있게 변환(mp3 파일-> 숫자 데이터-> 모델 입력)해주는 도구
- `.from_pretrained()` : 도구를 가져오는 함수

2. 로컬에서 음성을 텍스트로 변환하기

- (실습) 위스퍼 모델을 내려 받아 로컬에서 사용하기
 - 코드 설명
 - `pipeline()`
 - 입력 → 전처리 → 모델 실행 → 후처리 → 결과 출력 이 모든 과정을 자동으로 해주는 도구
 - 파라미터
 - `automatic-speech-recognition` : 음성 -> 텍스트 작업 지정
 - `tokenizer=processor.tokenizer` : 출력 & 일부 입력 담당(숫자 <-> 텍스트)
 - `feature_extractor=processor.feature_extractor` : 입력 담당(소리 -> 숫자)
 - `return_timestamps=True` : 시간 정보도 같이 반환
 - `chunk_length_s=10` : 긴 시간 10초씩 쪼개서 처리
 - `stride_length_s=2` : 2초씩 겹치게(0~10, 8~18, 16~20 ...)
 - 핵심 흐름
 - 오디오(mp3) -> `feature_extractor` -> 숫자 데이터 -> 모델 처리 -> 숫자 결과 -> `tokenizer` -> 텍스트 결과

2. 로컬에서 음성을 텍스트로 변환하기

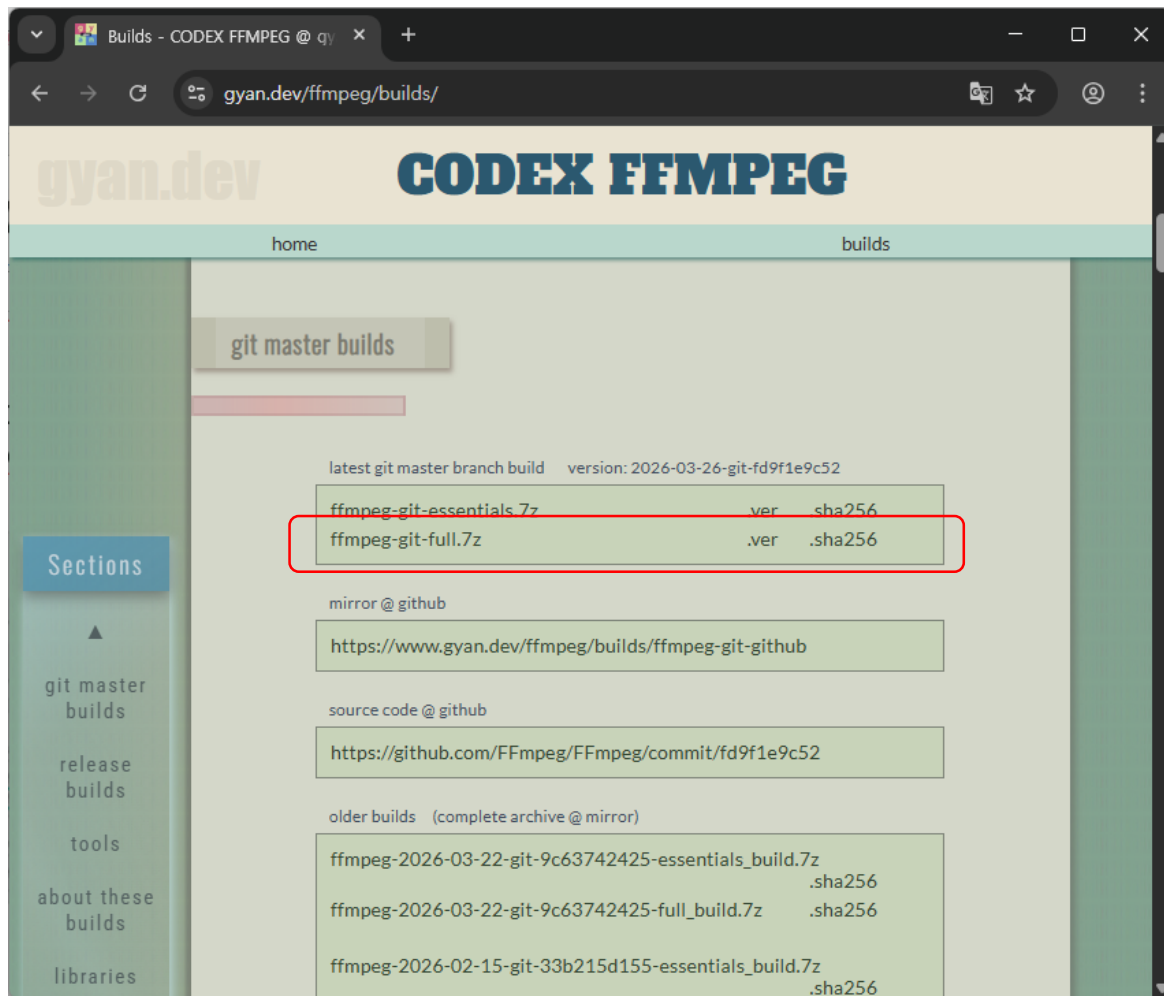
- (실습) 위스퍼 모델을 내려 받아 로컬에서 사용하기
 - 코드 작성 후 실행하면 error가 발생함
 - 허깅페이스의 whisper-large-v3-turbo 모델을 사용하려면 FFMPEG가 컴퓨터에 설치되어 있어야 함
 - FFMPEG는 오디오와 비디오 파일을 변환해 처리하는 오픈소스 도구
 - 예제 코드를 FFMPEG가 설치되지 않은 상태에서 실행하면 발생하는 오류

```
FileNotFoundError                                Traceback (most recent call last)
(... 생략 ...)
ValueError: ffmpeg was not found but is required to load audio files from filename
Output is truncated. View as a scrollable element or open in a text editor. Adjust
cell output settings...
```

2. 로컬에서 음성을 텍스트로 변환하기

- (실습) 위스퍼 모델을 내려 받아 로컬에서 사용하기

- FFMPEG 웹 사이트(<https://www.gyan.dev/ffmpeg/builds>)에 접속하여 ffmpeg-git-full.7z을 내려 받아 압축 풀기

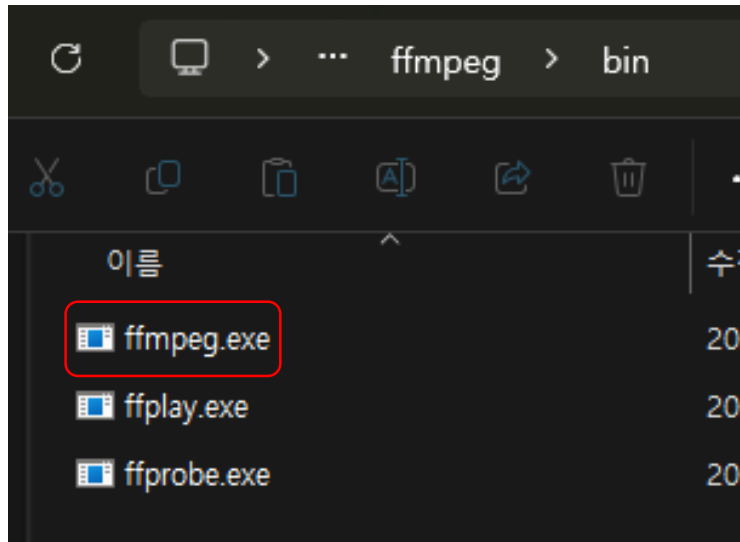


2. 로컬에서 음성을 텍스트로 변환하기

- (실습) 위스퍼 모델을 내려 받아 로컬에서 사용하기
 - 압축을 해제한 디렉토리 경로를 입력

```
import os

os.environ["PATH"] += os.pathsep + r"(압축을 해제한 디렉토리 경로)\ffmpeg\bin"
```



2. 로컬에서 음성을 텍스트로 변환하기

- (실습) 위스퍼 모델을 내려 받아 로컬에서 사용하기

- 실행 결과

```
{'text': ' 안녕하세요. 이 강의는 GPTAPI로 체법 만들기라는 내용을 다루는 강입니다. GPTAPI에 대해서 생소하신 분들도 있을 텐데 우리가 잘 알고 있는 ChatchBT 기능을 이용해서 우리가 원하는 프로그램을 어떻게 만드는데 대해서 이야기할 거예요. 그래서 이런 강의들이 사실 많이 있습니다. 여러 가지들이 있는데 이 강의 특징이라고 한다면 GPT로 명확한 미션을 사상하는 짜법 프로그램을 만드는 게 사실 쉽지는 않은데 이걸 어떻게 해서 구현을 하는지 그리고 그게 왜 필요한지에 대해서 좀 이야기를 할 거고요. 그 예제로 뭐 예전 여러 가지가 될 수 있는데 여기서 예제로 하는 것은 음악 플레이리스트 동영상을 자동으로 대화를 통해서 생성하는 프로그램을 만드는 것을 다룰 예정입니다. 그래서 프로그램이 실행되는 모습을 한번 보여드릴게요 우리가 만들 프로그램은 이런식으로 이제 나타나게 되고요', 'chunks': [{'timestamp': (0.0, 5.1), 'text': ' 안녕하세요. 이 강의는 GPTAPI로 체법 만들기라는 내용을 다루는 강입니다.'}, {'timestamp': (5.1, 8.68), 'text': ' GPTAPI에 대해서 생소하신 분들도 있을 텐데'}, {'timestamp': (8.68, 12.84), 'text': ' 우리가 잘 알고 있는 ChatchBT 기능을 이용해서'}, {'timestamp': (12.84, 16.0), 'text': ' 우리가 원하는 프로그램을 어떻게 만드는데 대해서 이야기할 거예요.'}, {'timestamp': (16.0, 18.0), 'text': ' 그래서 이런 강의들이 사실 많이 있습니다.'}, {'timestamp': (18.0, 24.84), 'text': ' 여러 가지들이 있는데 이 강의 특징이라고 한다면 GPT로 명확한 미션을 사상하는 짜법 프로그램을 만드는 게 사실 쉽지는 않은데'}, {'timestamp': (24.84, 28.3), 'text': ' 이걸 어떻게 해서 구현을 하는지 그리고'}, {'timestamp': (28.3, 33.28), 'text': ' 그 예제로 뭐 예전 여러 가지가 될 수 있는데 여기서 예제로 하는 것은 음악 플레이리스트 동영상을'}, {'timestamp': (33.28, 37.24), 'text': ' 자동으로 대화를 통해서 생성하는 프로그램을 만드는 것을 다룰 예정입니다.'}, {'timestamp': (37.24, 43.0), 'text': ' 그래서 프로그램이 실행되는 모습을 한번 보여드릴게요 우리가 만들 프로그램은 이런식으로 이제 나타나게 되고요'}]}
```

2. 로컬에서 음성을 텍스트로 변환하기

- (실습) 위스퍼 모델을 내려 받아 로컬에서 사용하기

- chunks에 저장된 값을 CSV 파일로 저장

```
# chunks를 CSV 파일로 저장
start_end_text = []

for chunk in result["chunks"]:
    start = chunk["timestamp"][0]
    end = chunk["timestamp"][1]
    text = chunk["text"]
    start_end_text.append([start, end, text])

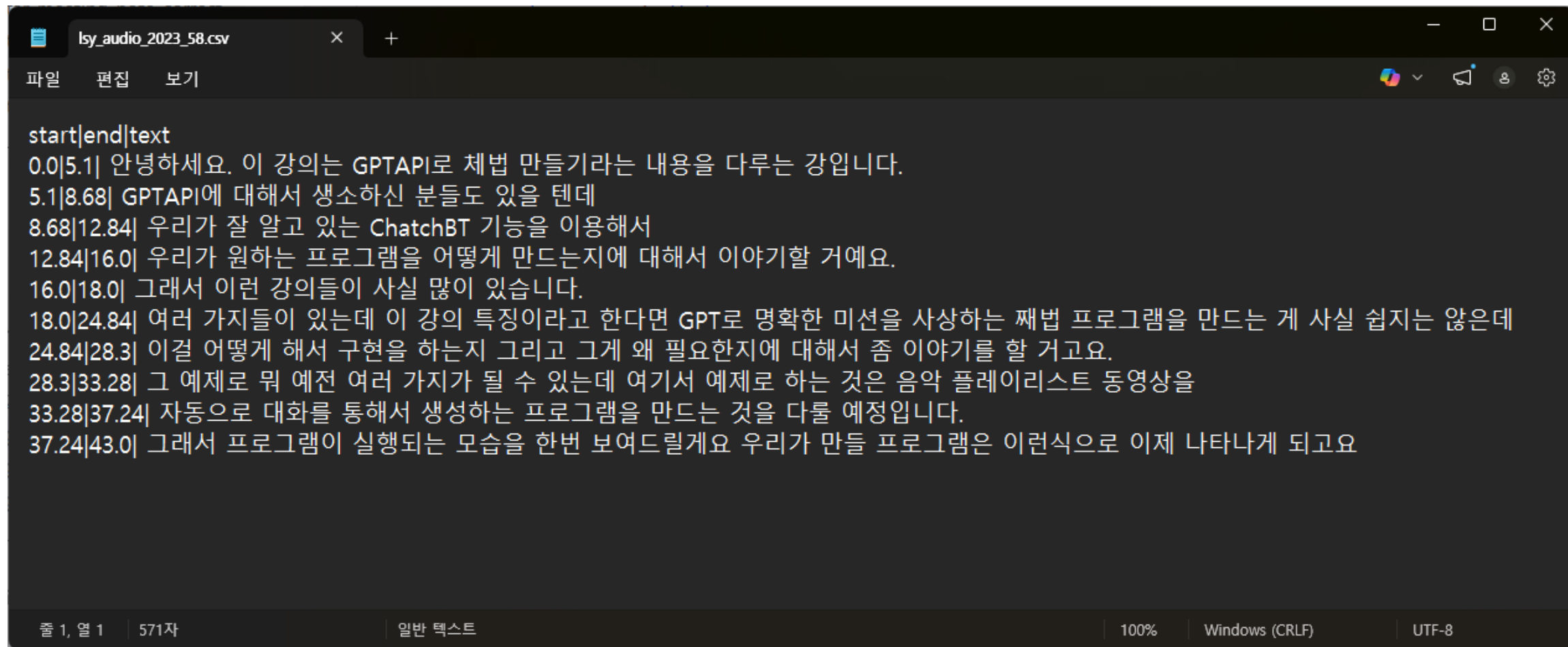
import pandas as pd
df = pd.DataFrame(start_end_text, columns=["start", "end", "text"])
df.to_csv("lsy_audio_2023_58.csv", index=False, sep="|")
print(df)
```



	start	end	text
0	0.00	5.10	안녕하세요. 이 강의는 GPTAPI로 체법 만들기라는 내용을 다루는 강입니다.
1	5.10	8.68	GPTAPI에 대해서 생소하신 분들도 있을 텐데
2	8.68	12.84	우리가 잘 알고 있는 ChatchBT 기능을 이용해서
3	12.84	16.00	우리가 원하는 프로그램을 어떻게 만드는데 대해서 이야기할 거예요.
4	16.00	18.00	그래서 이런 강의들이 사실 많이 있습니다.
5	18.00	24.84	여러 가지들이 있는데 이 강의 특징이라고 한다면 GPT로 명확한 미션을 사상하는 ...
6	24.84	28.30	이걸 어떻게 해서 구현을 하는지 그리고 그게 왜 필요한지에 대해서 좀 이야기를 할...
7	28.30	33.28	그 예제로 뭐 예전 여러 가지가 될 수 있는데 여기서 예제로 하는 것은 음악 플레...
8	33.28	37.24	자동으로 대화를 통해서 생성하는 프로그램을 만드는 것을 다룰 예정입니다.
9	37.24	43.00	그래서 프로그램이 실행되는 모습을 한번 보여드릴게요 우리가 만들 프로그램은 이런식...

2. 로컬에서 음성을 텍스트로 변환하기

- (실습) 위스퍼 모델을 내려 받아 로컬에서 사용하기
 - chunks에 저장된 값을 CSV 파일로 저장



```
start|end|text
0.0|5.1| 안녕하세요. 이 강의는 GPTAPI로 체법 만들기라는 내용을 다루는 강입니다.
5.1|8.68| GPTAPI에 대해서 생소하신 분들도 있을 텐데
8.68|12.84| 우리가 잘 알고 있는 ChatchBT 기능을 이용해서
12.84|16.0| 우리가 원하는 프로그램을 어떻게 만드는데 대해서 이야기할 거예요.
16.0|18.0| 그래서 이런 강의들이 사실 많이 있습니다.
18.0|24.84| 여러 가지들이 있는데 이 강의 특징이라고 한다면 GPT로 명확한 미션을 사상하는 짜법 프로그램을 만드는 게 사실 쉽지는 않은데
24.84|28.3| 이걸 어떻게 해서 구현을 하는지 그리고 그게 왜 필요한지에 대해서 좀 이야기를 할 거고요.
28.3|33.28| 그 예제로 뭐 예전 여러 가지가 될 수 있는데 여기서 예제로 하는 것은 음악 플레이리스트 동영상
33.28|37.24| 자동으로 대화를 통해서 생성하는 프로그램을 만드는 것을 다룰 예정입니다.
37.24|43.0| 그래서 프로그램이 실행되는 모습을 한번 보여드릴게요 우리가 만들 프로그램은 이런식으로 이제 나타나게 되고요
```